Отбор "LSM-дерево с нуля"

Алексеев Дмитрий Викторович

БПМИ-253 / ФКН ВШЭ

dmivikalekseev@edu.hse.ru

30 октября 2026 г.

Содержание

1 О себе **3**
1.1 Навыки/Опыт/известные алгоритмы 3
1.2 Чем вызван интерес 3

2 Существующие LSM-деревья **4**
2.1 Политика слияния 4
2.2 Some Well-Known Optimizations 5
2.2.1 Фильтр Блюма и подобные 5
2.2.2 Partitioning 6
2.3 LevelDB 6
2.4 RocksDB 6
2.5 Apache Cassandra / ScyllaDB 6
2.6 Apache HBase 6
2.7 CockroachDB 6
2.8 WiredTiger 6

3 Существующие Бенчмарки **6**
3.1 YCSB 6
3.2 db_bench 6

4 Бенчмаркинг **6**
4.1 Тесты RocksDB 6
4.1.1 Пермонас записи 6
4.2 Использованные VM: 8

Список литературы **10**

1 О себе

1.1 Навыки/Опыт/известные алгоритмы

Так как довольно просто выполнить тестовое на хорошем уровне из-за существования LLM, я вынужден написать подробный блок о себе

В данный момент я обучаюсь на потоке М+П+ ФКН ПМИ.

В силу отличной учебы на пилотном курсе АиСД-1 (Итоговая оценка: 9), оценка по АиСД-2 пока неизвестна, но я планирую так же получить отл. 9. я знаком с большинством стандартных продвинутых алгоритмов (в частности нужные для LSM дерева фильтр Блюма и Skip-List). Отдельно перечислю известные алгоритмы во внешней памяти: List ranking за $O(Sort(n))$, B/B^+-дерево, Куча во внешней памяти за $O(\frac{1}{B}\lceil \log_{M/B} \frac{q}{M} \rceil)$ на запрос, BFS во внешней памяти за $O(Sort(E) + V)$, поиск компонент связности за $O(Sort(E + V) \log V)$

Так же, я люблю много писать на C++ и достаточно хорошо им владею (закончил пилотный курс на 10, и продолжаю учится на продвинутом курсе сейчас). Есть опыт разработки больших проектов: написал свой интерепретатор для некоторого ЯП [1]. Так же, приведу ниже менее масштабные pet-проекты:

1. Интерактивные деревья поиска [2]
2. Программа для обработки фото [3] (вообще говоря, это был общекурсовой проект, но я тут заморочился с ООП, так что считаю необходимым это здесь прикрепить), p.s. все файлы сгружены в кучу, потому что тест система иначе не принимала

1.2 Чем вызван интерес

Мне очень понравились задачи во внешней памяти, которые нам предложили на курсе алгоритмов. И вообще меня заинтересовала разработка каких-то структур во внешней памяти, а данная тема для курсовой позволит мне реализовать данное желание

2 Существующие LSM-деревья

Данный раздел посвящён тому, какие вариации внутреннего устройства имеет LSM-дерево, для чего нужны те или иные модификации, а так же в каких реальных системах управления базами данных они используются.

Сначала давайте обсудим некоторые допущения, которые будем использовать в рассуждениях ниже. Источник [4] утверждает, что для LSM-дерева под стабильной нагрузкой (когда количество слоёв не меняется), перфоманс записи оптимален, когда для любого L максимальный количество ключей на $L+1$ уровне в T раз больше, чем максимальное количество ключей на L уровне.

2.1 Политика слияния

LSM-деревья различаются между собой по политике слияния дисковых компонент. [4] На практике обычно используют следующие две политики, каждая заточена под свой преобладающий тип запросов:

Leveling Merge Policy

Данная политика хранит компоненты следующим образом:

На каждом уровне будет хранится ровно одна компонента.

Максимальный объем компоненты на уровне L хранит в T раз больше ключей, чем компонента на уровне $L-1$. Тогда компонента на слое L сольется с компонентой на $L-1$ слое много раз (в точности T) перед тем, как переполнится и начнёт сливаться с компонентой на слое $L+1$.

Как итог, Leveling Merge Policy хранит "мало"компонент (по сравнению со второй политикой), благодаря чему, показывает лучший перфоманс на запросах чтения.

Tiering Merge Policy

Пусть вместимость $L-1$ слоя P - ключей, тогда эта политика, на каждом слое $L-1$ будет хранится не более T компонент, каждая размера $\frac{P}{T}$ с возможно пересекающимся диапазоном ключей. Когда на слое наберётся T заполненных компонент, сольем их в одну компоненту и положим на следующий уровень. Действительно, такое поведение будет корректным, потому что компонента на L слое вместит в себя T раз больше ключей, т.е. P, и максимальная вместительность уровня L в точности $P \cdot T$, что в T раз больше, чем на предыдущем уровне. И заполненный $L-1$ уровень станет ровно одной компонентой на L слое.

Эта политика, напротив, оптимизирована для запросов записи, за счёт снижения числа тяжёлых слияний, когда структуре приходится сливать старшие уровни.

Вообще говоря, классической политикой для LSM-дерева является именно Leveling Merge Policy, но параллельно LSM-дереву, Jagadish и др. в своей статье [5] представили очень похожую на LSMT структуру, оптимизированую под запросы записи, в которой применили "stepped-merge policy". Которая в последствии превратилась в Tiering Merge Policy, используемую в современных СУБД.

2.2 Some Well-Known Optimizations

Название данной секции дословно ссылается на название секции из статьи: "LSM-based Storage Techniques: A Survey"[4]

2.2.1 Фильтр Блюма и подобные

Фильтр Блюма - вероятностный алгоритм, для проверки принадлежности элемента множеству, гарантирующий отсутствие ложно-отрицательных срабатываний.

Устройство фильтра: есть фиксированное m - количество битов и k и k хэш функций $h_1, \ldots, h_k$, из некоторого семейства, $h_i : \mathcal{K} \to \{0, \ldots, m-1\}$. Положим масив A битов размера m. Когда в множество добавляется новый ключ K, положим $\forall i \in \{1, \ldots, k\} A[h_i(K)] := 1$. Проверка принадлежности ключа множеству проводится следующим образом: необходимо проверить, что $\forall i A[h_i(K)] == 1$.

Фильтр Блюма позволяет проверить отсутствие ключа в SSTable, перед началом дорого (с точки зрения числа I/O запросов) поиска ключа в самой таблице. В свою очередь, ложно-положительные срабатывания не влияют на корректность ответа, только лишь добавляют расходы на поиск внутри самой компоненты. Вероятность такого ложно-положительного срабатывания фильтра не превосходит $(1 - e^{-kn/m})^k$, тогда оптимальное количество хэш функций для минимизации ошибки достигается при $k := \frac{m}{2} \ln 2$, что на практике оказывается 10 и позволяет достичь 1% ложно-положительных срабатываний. А так как фильтр Блюма очень маленький и может быть загружен в оперативную память, накладные расходы на I/O запросы к диску во многих сценариях получается существенно сократить.

Собственно, идея о том, что с помощью некоторой небольшой вероятностной структуры можно с хорошей точностью быстро проверить отсутствие ключа (если ключа нет в таблице, то структура должна гарантированно это сообщить) в одной компоненте, находит применение во многих современных СУБД. LevelDB использует встроенный фильтр Блюма - NewBloomFilterPolicy [6]. A RocksDB позволяет выбирать фильтр: NewBloomFilterPolicy, NewRibbonFilterPolicy (Ribbon Filter - некото-

рый аналог фильтра блюма, который занимает меньше памяти). [7].

2.2.2 Partitioning

2.3 LevelDB

2.4 RocksDB

2.5 Apache Cassandra / ScyllaDB

2.6 Apache HBase

2.7 CockroachDB

2.8 WiredTiger

3 Существующие Бенчмарки

3.1 YCSB

3.2 db_bench

4 Бенчмаркинг

4.1 Тесты RocksDB

4.1.1 Пермонас записи

В данном подразделе проведём эксперименты по перфомансу записи "записей"с 16 байтовым ключом и 984 байтовым значением, в зависимости от способа организации SSTable и количества ядер предоставленных RocksDB. Сравнение проводится на виртуальной машине 1 во встроенном в RocksDB бенчмарке db_bench [8]. При следующих настройках:

```
$ db_bench \
          >out
          --benchmarks=fillrandom \
          --key_size=16\
          --value_size=984\
          --num= write_cnt / threads_cnt \
          --compaction_style=merge_policy \
          --threads=threads_cnt;
```

Тут представлен больше псевдокод общий для всех запусков. Все конфигурации количества потоков и политики слияния (compaction_style) запускаются на одном и том же число запросов случайной записи (с точностью на +-8).

В этом эксперименте мы будем тестировать перфоманс случайной записи в зависимости от количества ядер при разных политиках слияния.

compaction_style, =0 - реализует рассмотренную выше Leveling Merge Policy, а =1 - некоторая universal compaction, которая согласно документации RocksDB[9] "иногда называется tiered" , т.е. эта политика некотрый аналог выше рассмотренной Tiering Merge Policy.

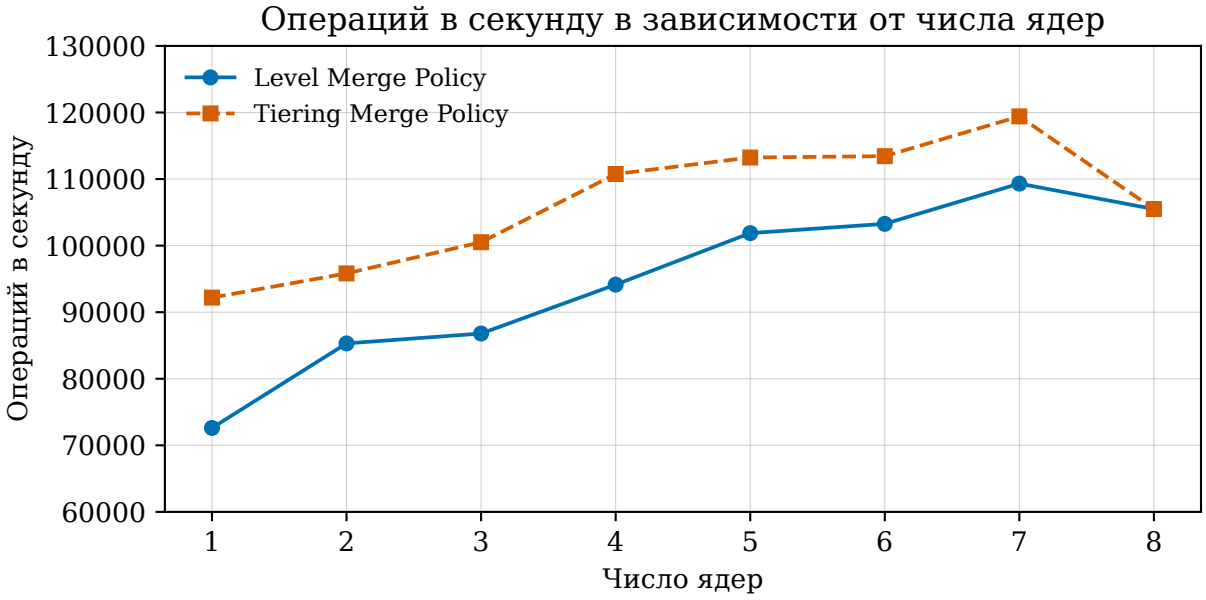


Рис. 1: Число записей в секунду при 10^7 операций случайной записи

Из графиков видно сразу несколько фактов:

Во-первых, не стоит отдавать СУБД все ядра системы, так как OS так же потребляет ресурсы компьютера, поэтому использование всех ядер ВМ 1 ведёт к снижению перфоманса.

Во-вторых, использование большего числа потоков записи всё ещё ведёт к увеличению числа операций, несмотря на необходимость синхронизации записи в LSMT (берём это как факт, т.к. так утверждают авторы статьи [4]).

В-третьих, Tiering Merge Policy-подобная система хранения, действительно показывает лучший перфоманс записи, который уже на трёх ядрах работает сопоставимо быстро семи (и предполагаю даже 8ю, если бы данные не были испорчены ОС) ядрам в режиме Level Merge Policy. При 4 ядрах Tearing MP показывает уже превосходящий перфоманс записи, чем 7-8 ядер Level MP.

Так как ключи точно умещаются в порядка 10-20GB, все или почти все операции могли проходить без участия внешней памяти, поэтому теперь давайте проведём более большой и долгий тест

Проведём повторный эксперимент, но уже сделав 10^8 операций случайных записей, суммарный размер записей на диске должен оказаться порядка 100GB

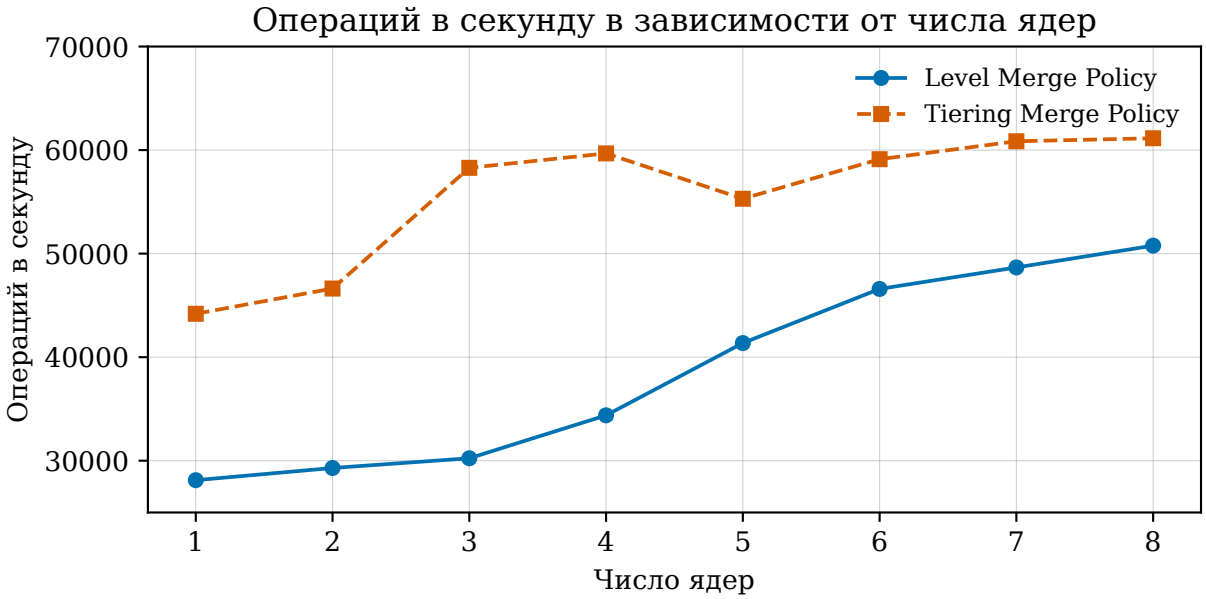


Рис. 2: Число записей в секунду при 10^8 операций случайной записи

Теперь видна серьезная разница скорости записи на диск между разными политиками. Действительно, Tiering MP справляется с "профильной"нагрузкой. А так же мы теперь видим замедление роста производительности с ростом числа потоков, теперь мы действительно уперлись в ресурсы накопителя и писать быстрее не можем

4.2 Использованные VM:

1. CPU:

   Model name: AMD EPYC-Genoa Processor

   CPU(s): 8

   Caches (sum of all):

   L1d: 128 KiB (4 instances)

   L1i: 128 KiB (4 instances)

   L2: 4 MiB (4 instances)

   L3: 32 MiB (1 instance)

   RAM:

   16GB (Более точная информация отсутствует из-за ограничения со стороны удаленной ВМ)

   SSD:

300GB (Подробные данные так же недоступны)

2. Чё те не нравится

Список литературы

1. dmitryAdams/bugguette. — URL: https://github.com/dmitryAdams/buguette (дата обр. 14.10.2026).
2. dmitryAdams/BST_garden. — URL: https://github.com/dmitryAdams/BST_garden (дата обр. 14.10.2026).
3. dmitryAdams/image_processor. — URL: https://github.com/dmitryAdams/image_processor (дата обр. 14.10.2026).
4. *Luo C.*, *Carey M. J.* LSM-based storage techniques: a survey: C. Luo, MJ Carey // The VLDB Journal. — 2020. — Т. 29, № 1. — С. 393—418.
5. Incremental organization for data recording and warehousing / H. Jagadish [и др.] // VLDB. — ResearchGate GmbH. 1997. — С. 16—25.
6. google/leveldb. — URL: https://github.com/google/leveldb (дата обр. 25.10.2026).
7. facebook/rocksdb. — URL: https://github.com/facebook/rocksdb/ (дата обр. 25.10.2026).
8. Benchmarking tools. — URL: https://github.com/meta/rocksdb/wiki/Benchmarking-tools (дата обр. 28.10.2026).
9. RocksDB - Wikipedia. — URL: https://en.wikipedia.org/wiki/RocksDB (дата обр. 25.10.2026).